JavaScript Technical Interview Answer

Senior Developer Assessment & Reference Guide

QUESTION

> **Pass by value আর pass by reference এর পার্থক্য কী?**

SHORT INTERVIEW ANSWER

In **Pass by Value**, a copy of the actual primitive data value is passed to the function, so modifications inside the function do not affect the original variable. In **Pass by Reference**, a memory reference address of the variable is passed, meaning changes inside the function directly mutate the original object in memory. Strictly speaking, JavaScript is **always pass-by-value**; when passing objects, JS passes the *copy of the reference address* (often called pass-by-sharing).

DETAILED EXPLANATION

Understanding value vs reference passing is fundamental to memory management and state handling in JavaScript:

- **Primitive Types (Pass by Value):** Primitive values (`Number`, `String`, `Boolean`, `Undefined`, `Null`, `Symbol`, `BigInt`) are stored directly on the **Call Stack**. When passed into a function or assigned to another variable, JS allocates a new memory slot on the stack and copies the bit-for-bit raw value.
- **Non-Primitive Types / Objects (Call by Sharing / Passed-by-Value-Reference):** Complex types (`Object`, `Array`, `Function`) reside in the **Memory Heap**. The variable on the call stack holds a *pointer (address)* pointing to that heap memory address. When passed to a function, JS copies the pointer value onto the new call stack frame.
- **Why mutations reflect:** Since both the outer variable and the function parameter store copies of the exact same memory pointer, mutating properties via `param.prop = newValue` modifies the underlying object on the heap.
- **Why reassignments don't affect outer variables:** Overwriting the inner variable with a new reference (e.g., `param = { newObj: true }`) breaks the link by changing the internal pointer without affecting the outer pointer variable.

Feature	Pass by Value	Pass by Reference (Sharing)
Data Types	Primitives (Number, String, Boolean, etc.)	Objects, Arrays, Functions
Memory Allocation	Stored directly in Call Stack	Heap memory, referenced by Call Stack pointer
Function Arguments	Copies actual primitive value	Copies the memory reference pointer
Outer State Effect	Changes inside function NEVER affect outer variable	Property mutations affect outer object; total reassignment does not

SYNTAX

```
// Passing primitives (Pass by Value)
function fnPrimitive(val) { val = 100; }

// Passing objects (Pass by Copy of Reference / Pass by Sharing)
function fnObject(obj) { obj.prop = "updated"; }
```

EXAMPLE

```
// 1. Primitive Example (Pass by Value)
let score = 50;

function updateScore(val) {
  val = 100; // Modifies local variable on stack frame
}

updateScore(score);
console.log("Score:", score); // 50 (Unchanged)

// 2. Reference Type Example (Pass by Copy of Reference)
const user = { name: "Alice" };

function updateUser(obj) {
  obj.name = "Bob"; // Mutates shared heap memory
}

updateUser(user);
console.log("User Name:", user.name); // "Bob" (Mutated)
```

OUTPUT

```
Score: 50
User Name: Bob
```

COMMON INTERVIEW FOLLOW-UP QUESTIONS

1. Is JavaScript technically a 100% pass-by-value language?
2. What happens if you reassign an object inside a function (e.g., `obj = {}`)?

3. How does memory stack vs heap allocation work in relation to value and reference types?
4. How can you pass an object to a function safely without allowing mutation?

COMMON MISTAKES

- Assuming JavaScript supports true "pass-by-reference" like C++ aliases (`&var`).
- Expecting reassignment inside a function (`obj = { newKey: "val" }`) to update the original reference outside.
- Forgetting that array operations like `push()` or `splice()` mutate the original array passed as an argument.

BEST PRACTICES

- Treat parameters as immutable; avoid mutating object arguments directly to keep functions pure.
- Pass shallow or deep clones (e.g., `structuredClone()` or spread `{...obj}`) when modifications should not affect external state.

REAL-WORLD USE CASE

In state management libraries like React and Redux, understanding pass-by-value/reference is essential. React relies on reference equality (`prevProps.obj === nextProps.obj`) to detect state updates and trigger re-renders. Accidentally mutating an object directly fails to change its reference, leading to silent bugs where components do not re-render.